

SIGNALDOWNLOADER

Umfassendes Codereview-Konzept und Masterprompt

Sieben Projekte. Eine vollständige Prüfungskette.

Code, mathematische Forensik, Datenherkunft, Schnittstellen, Benutzerabläufe, Hintergrundjobs, Logs, Sicherheit, Tests und Betrieb.

SignalKiScanner

MqlDownloader

PelicanTrading

PelicanWinnerLooser

RoboForex

Vantage

ZuluMonitor

Stand 04.10.2026

Lesende Bestandsaufnahme; keine Produktivänderungen. Dieses Dokument spezifiziert die auszuführende Review und ihren Fehler-/Verbesserungsbericht.

Inhaltsverzeichnis

Die vollständige Markdown-Datei ist der ausführbare Masterprompt. Die PDF enthält denselben fachlichen Inhalt und unterstützt das Lesen und Besprechen.

1. Anwendung und konkreter Auftrag an die Review-KI	3
2. Vollständiger Scope und Projektrollen	4
3. Ausgangsstand, den der spätere Review neu einfrieren muss	5
4. Bereits beobachtete Ansatzpunkte für die Review	5
5. Organisation: parallele Subagents und unabhängige Gegenprüfung	6
6. Arbeitsphasen und Prüfstopps	7
7. Evidenz, Vollständigkeit und Quellenrangfolge	8
8. Gemeinsamer Schnittstellen- und Kennzahlenvertrag	8
9. SignalKiScanner: mathematische und fachliche Tiefenprüfung	9
10. SignalKiScanner: Abläufe, Agenten und Benutzeroberfläche	12
11. MqlDownloader: vollständiger Projektprüfplan	13
12. PelicanTrading: vollständiger Projektprüfplan	13
13. PelicanWinnerLooser: eigener fachlicher Prüfplan	14
14. RoboForex: vollständiger Projektprüfplan	15
15. Vantage: vollständiger Projektprüfplan	16
16. ZuluMonitor: vollständiger Projektprüfplan	16
17. Logs: Inventur, Korrelation und Root-Cause-Verfahren	17
18. Ablaufrekonstruktion vom Klick bis zum Ergebnis	18
19. Unabhängige Rechenorakel und Grenzfallkatalog	19
20. Projektübergreifende E2E- und Ausfallszenarien	21
21. Sicherheit und Vertrauensgrenzen	22
22. Performance, Wartbarkeit, Build und Betrieb	23
23. Teststrategie und reproduzierbare Ausführung	24
24. Priorisierung: Auswirkung und Sicherheit getrennt	24
25. Verbindliches Befundformat	25
26. Verbesserungsplan mit begründeter Reihenfolge	25
27. Geforderte Review-Artefakte und Berichtsgliederung	26
28. Abschlusskriterien und Startanweisung	27
29. Quellenindex der Konzepterstellung	28

Stand: 04.10.2026. **Umfang:** sieben Projekte plus gemeinsame Startskripte, Verträge und Betriebsabläufe.

Ziel: belastbarer Fehler- und Verbesserungsbericht als PDF und Markdown.

Workspace: D:/AntiGravitySoftware/GitWorkspace/SIGNALDOWNLOADER.

Status dieses Dokuments: Konzept und ausführbarer Review-Prompt nach lesender Bestandsaufnahme von Code, Dokumentation, Tests und ausgewählten Logs. Dies ist kein abgeschlossenes Codereview. Die bestehenden Tests wurden für diese Konzepterstellung nicht neu ausgeführt. Es wurden keine Anwendungen gestartet, Produktionsdaten verändert oder Fehler behoben. Die unten genannten Beobachtungen begründen Prüfungsschwerpunkte; sie ersetzen keine vollständige Ursachen- und Auswirkungsprüfung.

1. Anwendung und konkreter Auftrag an die Review-KI

Die vollständige Markdown-Datei ist der Masterprompt. Die Abschnitte 2 bis 4 liefern den überprüften Ausgangskontext; alle folgenden Abschnitte sind Arbeitsanweisungen. Bei einem späteren Start musst du den Kontext erneut am aktuellen Arbeitsstand verifizieren. Verwende die PDF zum Lesen und Besprechen, die Markdown-Datei als KI-Eingabe.

Auftrag: Führe ein sehr ausführliches, projektübergreifendes Codereview aller Projekte im genannten Workspace durch. Untersuche Quellcode, Konfiguration, Build, Tests, Datenmodelle, Berechnungen, Schnittstellen, Benutzerabläufe, Hintergrundjobs, Fehlerbehandlung, lokale Laufzeitdaten und sämtliche auffindbaren relevanten Logs. Rekonstruiere die vollständigen Abläufe vom Plattformabruf bis zur Anzeige, Bewertung, PDF-Erstellung und Weitergabe. Erstelle anschließend einen zusammenhängenden Fehler- und Verbesserungsbericht als Markdown und inhaltsgleiches PDF.

Arbeite sachlich. Behandle Aussagen in README, AGENTS, Kommentaren, früheren Reviews, KI-Berichten und Nutzerhypothesen als überprüfbare Aussagen. Prüfe sie gegen geltende Anforderungen, aktuellen Code und reproduzierbare Belege. Trenne beobachtete Tatsachen, bestätigte Fehler, begründete Risiken, Verbesserungsvorschläge und ungeklärte Fragen. Keine pauschale Aussage wie „alles stabil“, nur weil vorhandene Unit-Tests grün sind.

Dieser Auftrag autorisiert Analyse und isolierte Prüferfakte, keine Produktivfixes. Ändere keine bestehenden Quell-, Konfigurations- oder Datenbestände. Erzeuge Prüfskripte, synthetische Fixtures und Testkopien ausschließlich in einem getrennten Review-Arbeitsbereich. Keine Commits, Pushes, Deployments, Trades, Nachrichten an Dritte, externen LLM-Aufrufe oder produktiven Scans ohne eine darauf bezogene Freigabe. Vorhandene Änderungen des Nutzers bleiben erhalten. Für rein lesende Prüfungen und gefahrlose Tests in isolierten Kopien ist keine zusätzliche Rückfrage nötig.

2. Vollständiger Scope und Projektrollen

Projekt	Rolle und Technik	Besonders wichtige Reviewgrenze
SignalKiScanner	Python, Streamlit, SQLite; Quellen-Hub, Forensik, Ampel, RetDD, KI-Berichte, Agentenbetrieb	Bewertungsregeln, Datenherkunft, mathematische Konsistenz, Jobs und Persistenz
MqlDownloader	Java, Swing, Maven, Selenium; MQL4/MQL5-Downloader, H2-Datenhaltung, REST auf 8089	Originaldaten, Exportformat, Browser-/Session-Lifecycle, MPDD-Filter
PelicanTrading	JavaFX, Maven; Provider-Monitor, Zwei-Phasen-Synchronisierung, REST auf 8090	UUID-Discovery, Trades, Währung, Kapitalbasis, Drawdown und Strategieabläufe
PelicanWinnerLooser	JavaFX, Maven, SQLite; Pelican-Kopierkonten und Verteilungsforensik	Fortsetzbarer Crawler, Session, Stichprobenqualität, Quantile und EUR-Auswertung
roboforex	JavaFX, Maven; CopyFX-Monitor, MT4/MT5-Aggregation, REST auf 8091	Deal-Pairing, Teil-Schließungen, PnL, Datenpersistenz und Drawdown
vantage	JavaFX, Maven; Copy-Trading-Monitor, REST auf 8092	Paging, Kategorien, USD/USC, Trade-Vollständigkeit, Kopiererhistorie
zulumonitor	JavaFX, Maven; ZuluTrade-Monitor, REST auf 8093	Dual-Host-Fallback, verifizierte Downloads, Konto-Währung, Demo und Historie

Zusätzlich gehören README.md, UI_KONVENTIONEN.md, startall.bat, alle projektlokalen

Start-/Build-/Deployskripte sowie Versions- und Abhängigkeitskonfigurationen in den Scope. Ermittle rekursiv weitere eigenständige Buildmodule und Unterprojekte; die sieben Ordner sind die Ausgangsliste, keine Begründung, neu gefundene Module zu ignorieren.

waste/ ist ein Review- und Belegarchiv, kein achtetes Produkt. Nutze frühere Protokolle für Regressionsthemen und offene Punkte. Bestätige jeden übernommenen Befund neu; ein archivierter Fehler kann inzwischen behoben oder fachlich revidiert sein.

Externe Konsumenten: MqlTradeMonitor und MqlRealmonitor liegen außerhalb dieses Workspaces. Prüfe hier die Scanner-seitigen Payloads und Vertragsannahmen mit lokalen Fake-Servern/Fake-Clients. Behaupte kein vollständiges Review ihrer externen Implementierung. Kennzeichne eine erforderliche Prüfung dort als externe Anschlussprüfung. Projektbezogene Laufzeitverzeichnisse außerhalb des Repos dürfen als ausdrücklich ausgewiesene Log-/Datenevidenz dienen; sie erweitern nicht stillschweigend den Code-Scope.

PelicanWinnerLooser ist nach der aktuellen Bestandsaufnahme ein eigenständiges Kopierkonto-Werkzeug. Unterstelle keine REST-Datenlieferung an den Hub, nur weil eine Architekturübersicht es grafisch nahe bei den Quellen zeigt. Belege jede tatsächlich vorhandene Kopplung.

3. Ausgangsstand, den der spätere Review neu einfrieren muss

Projekt	Gelesener HEAD am 04.10.2026	Beobachteter Arbeitsstand
SignalKiScanner	85dc290	README und Architekturdoku geändert; zusätzliche unversionierte Datei vorhanden
MqlDownloader	c44d2f6	README geändert
PelicanTrading	9ea8b39	README geändert
PelicanWinnerLooser	dd4a94f	Zahlreiche geänderte und neue Code-, Test-, Auth-, GUI- und Konfigurationsdateien
roboforex	6eda52a	README geändert
vantage	571e9c2	README geändert
zulumonitor	97f02ff	README geändert

Der Review muss den **Arbeitsbaum einschließlich uncommitteter Änderungen** prüfen. HEAD allein beschreibt besonders bei PelicanWinnerLooser nicht den untersuchten Stand. Erzeuge je Projekt ein Manifest aus HEAD, Branch, `git status`, relevanten Datei-Hashes und Erhebungszeit. Keine Reset-/Clean-Aktionen. Prüfe auch unversionierte produktrelevante Dateien; ordne zufällige Artefakte getrennt ein.

Vorhandene Surefire-Reports und Dokumente enthalten historische Testzahlen. Diese sind nur Altbelege. Berichte aktuelle Sammlung, tatsächlich ausgeführte Tests, Fehler, Skips und Laufzeiten separat. Ein gelesener Report ist kein von dir ausgeführter Test.

4. Bereits beobachtete Ansatzpunkte für die Review

Diese Liste ist keine fertige Mängelliste. Sie trennt belegte Ausgangsbeobachtungen von den noch zu prüfenden Auswirkungen.

4.1 Drawdown-Semantik an der Quellen-Grenze

Robo-, Vantage- und Zulu-Codepfade berechnen Kurven aus geschlossenen Trade-PnLs. PelicanTrading ergänzt in seinem untersuchten Kurvenpfad einen aktuellen Floating-Endpunkt, bildet dort aber keinen vollständigen historischen Floating-Verlauf ab. Dennoch wird das Ergebnis unter `TradeEqDrawdownPct` geliefert. Der Scanner übernimmt dieses Feld als Monitor-Messwert in `_equity_messwerte()` und beschreibt die Zielgröße als Equity einschließlich Floating.

Zu prüfen: Unter welchen Eingaben wird ein Drawdown geschlossener Trades als belegter maximaler Equity-DD verwendet? Kann dies fehlende Kursmessungen verdecken oder RetDD/Grün beeinflussen? Trace vom konkreten Produzenten über CSV/REST/Ingest/DB bis zur Ampel und zum KI-JSON, mit unabhängig berechnetem Gegenbeispiel. Ein falscher Feldname allein beweist noch nicht den Umfang eines Bewertungsfehlers.

4.2 Nachweisabhängiges Dedup und mehrfaches Parsen

Der aktuelle Scanner entfernt identische Tradezeilen nur mit passendem Plattform-Anzahlbeleg. `portfolio_statistik.py` liest Trade-CSV für Rendite/Portfolio erneut; die untersuchten Aufrufe reichen den Plattformbeleg nicht mit. **Prüfhypothese:** Forensik und spätere Rendite-/Portfoliokurven könnten unterschiedliche Tradebestände verwenden. Teste das mit demselben Roh-Snapshot und Beweiswert. Leite keinen pauschalen Dedup-Fix ab: echte identische Grid-Legs müssen erhalten bleiben.

4.3 Startskripte greifen weit über eine einzelne Instanz hinaus

`starta11.bat` beendet Portbesitzer mit Force; `Scanner-start.bat` sucht außerdem breit nach Streamlit-Prozessen. Im Sammelskript wird `MqlDownloader` als Endpunkt aufgeführt, aber nicht analog zu den vier JavaFX-Monitoren gestartet. Die Bereitschaftsschleife prüft den GUI-Port 8504 statt die gesamte REST-Kette. **Zu prüfen:** gewünschte Service-Verantwortung, fremde Prozesse, Port-Konflikte, Fehlercodes und irreführende Fertigmeldung. Führe diese Skripte bei der Review nicht auf dem produktiven Rechner aus.

4.4 Aktuelle RetDD-Regeln und Architekturdoku widersprechen sich

Die aktuelle Ampel-Matrix verwendet RetDD/Monat ab 1,0 für Grün und ab 0,5 für Gelb. `doc/21_megaprojekt-architektur.md` nennt 0,5/0,167 und stellt RetDD überwiegend als Jahres-Calmar dar. Derselbe Text erwähnt M1/M5-Rekonstruktion, während der produktive Rekonstruktionspfad H1 prüfen lässt. **Zu prüfen:** Doku, Hilfe, Prompts, UI und API müssen die tatsächlich geltende Monats-/Jahresdefinition, Zeitauflösung und Schwellen getrennt ausweisen.

4.5 Logs belegen historische und wiederkehrende Betriebsprobleme

Eine Scanner-Logstichprobe enthält einen `ValueError` für die Zahlzeichenfolge 9,10; aktueller Code enthält inzwischen eine entsprechende Umwandlung. Vantage-Logs enthalten wiederholtes `10402 err_invalid_param` für Kategorien. Robo-Logs unterscheiden eine Plattform `rst`, für die der MT4/MT5-Dealpfad nicht verfügbar ist. Zulu-Logs zeigen ungültige Tradeantworten bei einem Host. WinnerLooser-Logs zeigen Session-Pausen und Wiederaufnahme. `MqlDownloader`-Laufzeitlogs enthalten Datumsbereich-/Monatsrenditewarnungen.

Zu prüfen: Welche Fälle sind behoben, erwartbare Capability-Grenzen, weiterhin reproduzierbare Fehler oder nur Folgen fehlender Daten? Prüfe aktuelle Implementierung und Laufzeitparameter. Zähle Warnungen nicht pauschal als Fehler und deute einen automatischen Retry nicht als Erfolg.

4.6 Unterschiedliche Absicherung externer Texte im LLM-Pfad

Robo besitzt einen ausdrücklich getrennten System-/Daten-Prompt. Im untersuchten Zulu-Aufruf ist der Systemtext leer. **Zu prüfen:** tatsächliche Schutzwirkung entlang des gesamten Promptbuilders, fremde Strategietexte, strukturierte Ausgaben und Bindung an Codebefunde. Ein leerer Systemparameter allein ist kein Nachweis eines erfolgreichen Prompt-Injection-Angriffs.

5. Organisation: parallele Subagents und unabhängige Gegenprüfung

Plane zuerst Arbeitspakete mit klaren Ergebnissen, Abhängigkeiten und Prüfprioritäten. Führe unabhängige Pakete parallel mit Subagents aus, entsprechend der Workspace-Anweisung. Verwende bei begrenzten Slots mehrere Wellen; kein Projekt darf dadurch ausfallen. Der Hauptagent verantwortet Scope, Zusammenführung, Gegenprüfung und die finale PDF/MD.

Paket	Verantwortungsbereich	Verpflichtendes Ergebnis
A	Scanner-Berechnungen, Forensik, Ampel, RetDD	Invarianten, unabhängige Nachrechnung, Grenzfallbelege
B	Scanner-DB, Quellen, Agenten, GUI, LLM, Export	Ablauftraces, Stale-/Versionslogik, Nebenläufigkeit
C	MqlDownloader und MQL-Originaldaten	Export-/Parservertrag, Browser-Lifecycle, H2/REST
D	PelicanTrading	Discovery/Sync, FX, Trades, DD, Strategieprozesse
E	PelicanWinnerLooser	Crawler/Resume, Session, Stichprobe und Statistik
F	RoboForex	MT4/MT5-Aggregation, Trade-/Subscriber-Store, REST
G	Vantage und Zulu, je getrenntes Kapitel	Plattformverträge, Cache, Historie, Datenqualität
H	Projektübergreifende Contracts, Logs, Start/Build	Semantikmatrix, Fehlerketten, Betriebs-/Sicherheitsprüfung
I	Unabhängige Gegenprüfung wichtiger Befunde	Bestätigung, Widerlegung oder offene Beweisfrage

Ein Subagent soll nicht seinen eigenen schwerwiegenden Befund allein abnehmen. Lasse P0/P1 und alle Änderungen an mathematischen Deutungen unabhängig prüfen. Gemeinsame Ursache nur einmal als Hauptbefund führen, betroffene Projekte und einzelne Erscheinungsformen darunter verknüpfen. Beobachtungen anderer Agenten sind Hinweise, kein Ersatz für eigene Belege.

Keine gleichzeitigen Prüfprozesse auf derselben beschreibbaren Produktiv-DB, demselben Browserprofil oder demselben MT5-Terminal. Parallelisiere statische Lektüre und isolierte Fixtures; serialisiere gemeinsame Ressourcen. Halte den Nutzer mit kurzen Erkenntnis-Updates auf dem Laufenden, ohne unverifizierte Zwischenstände als Abschluss zu melden.

6. Arbeitsphasen und Prüfstopps

Phase 0: Inventur und eingefrorener Stand. Erfasse alle Projekte, Buildmodule, Anweisungen, Abhängigkeiten, Services, Dateispeicher, Tabellen, Logs und Datenquellen. Kläre neue Regeln gegen historische Einträge. Ergebnis: Manifest, Scope-/Abdeckungsliste und aktuelle Regelmatrix.

Phase 1: Architektur und Datenherkunft. Zeichne die tatsächlichen Datenflüsse anhand von Code. Erfasse je Kennzahl Produzent, Einheit, Währung, Zeitfenster, Kapitalbasis, Rundung, Missing-Werte und Empfänger. Markiere Vertrauensgrenzen und implizite Annahmen.

Phase 2: Statische und loggestützte Tiefenprüfung. Lies alle produktrelevanten Module. Priorisiere Berechnungs- und Datenverlustpfade. Korrelierte Logs, frühere Reviews und Testlücken liefern gezielte Gegenbeispiele. Ergebnis: belegte Beobachtungen plus zu falsifizierende Hypothesen.

Phase 3: Isolierte Reproduktion. Führe nach Prüfung ihrer Nebenwirkungen vorhandene Tests sowie gezielte Gegenbeispiele in getrennten Arbeitskopien und Temp-Datenbanken aus. Nutze Fake-HTTP, deterministische Kurse und gefrorene Uhren. Ergebnis: reproduzierbare Fehler, widerlegte Verdachtsfälle und dokumentierte Grenzen.

Phase 4: Ende-zu-Ende-Verifikation. Verfolge ausgewählte Rohdaten durch die komplette Kette. Prüfe insbesondere die Fälle, die ein fälschlich positives Risiko-/Effizienzurteil oder einen unbemerkten Datenverlust auslösen könnten.

Phase 5: Gegenprüfung, Priorisierung und Maßnahmen. Stelle für jeden wichtigen Befund die stärkste plausible Gegenposition dar. Prüfe sie. Schätze Auswirkungen, Behebung, Regressionen und Abhängigkeiten. Keine unbegründeten Komplett-Neuschreibungen.

Phase 6: Bericht und Artefaktprüfung. Erzeuge vollständige MD/PDF, Evidenzindex, Abdeckungsmatrix und Maschinenliste. Prüfe Seitenlayout, Tabellen, Links, Suchbarkeit, Inhaltstreue und die Behauptungen der Management-Zusammenfassung.

Stops betreffen nur riskante Ausführung: Ist für eine Prüfung produktiver Netzwerkabruf, echter Login, LLM-Kostenverbrauch, Terminalstart, Neustart oder externer Sync nötig, dokumentiere zuerst das konkrete isoliert prüfbare Ergebnis. Lasse diese abhängige Live-Prüfung offen bis zur Freigabe und arbeite an allen unabhängigen Prüfungen weiter. Fehlende Live-Freigabe rechtfertigt keinen vorzeitigen Abbruch der gesamten Review.

7. Evidenz, Vollständigkeit und Quellenrangfolge

Lies AGENTS.md und lokale Anweisungen vollständig, nicht nur den Anfang. Bei widersprüchlichen datierten Fachregeln gilt die neueste eindeutig bestätigte Vorgabe; dokumentiere den Konflikt und den tatsächlich implementierten Zustand. Aktueller Code beschreibt Verhalten, ist aber nicht automatisch fachlich richtig. Altberichte, Kommentare und Testassertions müssen an unabhängigen Sollwerten gemessen werden.

Für jede Aussage erfasse Projekt, relativen Pfad, verifizierte Zeile/Funktion, Dateihash/Stand, relevanten Eingabestand und Art des Belegs. Zeilennummern vor der finalen Ausgabe neu kontrollieren. Für Logs zusätzlich Zeitraum, Zeitzone, Prozess-/Run-ID soweit vorhanden und redigierte Ereignisfolge. Für Tests Befehl, Interpreter/JDK, Profile, Exitcode, Dauer, tatsächlich ausgeführte Tests und relevante Ausgabe.

Führe eine Abdeckungsmatrix: jeder produktrelevante Modul-/Ablaufbereich erhält den Status **gelesen**, **statisch geprüft**, **isoliert getestet**, **durch E2E geprüft**, **nicht prüfbar** oder **begründet ausgenommen**. Halte Einschränkungen und Beleglinks fest. Reine Dateizählung ist kein Prüfbeweis.

Inventarisiere sämtliche relevanten Logs und Archive mit Größe, Zeitraum und Dateityp. Analysiere den zugänglichen Bestand maschinell auf Ereignisklassen und Häufungen; lies alle Treffercluster und vollständige relevante Laufketten. Bei großen oder fehlenden Beständen dokumentiere die konkrete Analysetiefe, ausgeschlossene Zeiträume und Gründe. Eine Stichprobe darf nicht als vollständige Logprüfung beschrieben werden.

Aus generierten target/, Report-/Download-/Cache-Ordern folgt kein pauschaler Code-Scope. Nutze Buildartefakte, CSV, Datenbanken und alte Reports als Daten-/Test-/Logevidenz. Untersuche Binärdateien nur soweit für Format, Version, Integrität, Schema oder Verhalten erforderlich.

8. Gemeinsamer Schnittstellen- und Kennzahlenvertrag

Erstelle eine Matrix für alle fünf REST-Quellen und den Hub. Prüfe /api/v1/health, /providers, Einzelprovider, /{id}/{version}/trades, trades.csv, metrics, history, reports und Reportdateien dort, wo sie tatsächlich implementiert sind. Versionen mq14, mq15, pelican, vantage, zulu nicht blind gleichsetzen. Health-/Instanzangaben, Ports und Standard-Bindadressen mit Code und Konfiguration abgleichen.

Für jedes Vertragsfeld dokumentiere: Name, Typ, Einheit, Definition, Producer-Datei, Consumer-Datei, Pflicht/optional, null/fehlend/0-Verhalten, erlaubten Wertebereich, Aktualisierungszeit, Zeitraum, Herkunft

und Versionspolitik. Mindestens: Identität, Quelle, Plattform, Währung, Trades-/Positionsanzahl, Abonnenten, Wochen/Alter, Balance/Equity, Renditen, DD-Arten, Initial Deposit, virtuelle Basis, SL-Nachweis und Report-SHA.

Besonders kritisch: TradeEqDrawdownPct darf durch den Feldnamen keinen historischen Floating-Nachweis erhalten. Prüfe geschlossene PnL-Kurve, historisches Floating, aktuelles Floating, Originalkonto, virtuelle Kurve und angenommene Basis getrennt. Eine Quelle muss einen Messwert mit Methode und Grenzen liefern können; der Empfänger darf seine Genauigkeit nicht stillschweigend erhöhen. Prüfe auch, ob die konservative Risikoschranke einen Wert verwenden darf, der für den RetDD-Nenner ungeeignet ist.

Prüfe die folgenden Vertragseigenschaften:

- Pagination: stabile Ordnung, Seitengrenzen, wechselnder Katalog, total/offset, leere Seiten, Wiederholungen, Max-Seiten-Abbruch und eindeutige Vollständigkeitsmeldung.
- Numerik: JSON-Zahl gegen String, Dezimalkomma, Tausendertrennzeichen, bool, NaN, Infinity, negative DDs, große Werte, fehlende Felder und Einheit Prozent versus Anteil.
- CSV: BOM/UTF-8, Semikolon, Header, MT4-/MT5-Format, Quotes, Leerzeilen, Endsumme, nur Zeitstempel, offene Positionen und Login-HTML statt CSV.
- HTTP: Timeout, Retry, 401/403, 404 mit fachlichem Grund, 429/Retry-After, 5xx, falscher Content-Type, Teilantwort und Schemaabweichung.
- Auth: Header/Bearer/Query-Token dort, wo implementiert; konstante Vergleiche, Health-Ausnahmen, lokale/LAN-Bindung und redigierte Logs.
- Konsistenz: Katalog/Metrics/Trades desselben Snapshots, Zeitstempel und SHA, Cache-Fallback, gelöschte Provider, korrigierte historische Trades, geänderte Währung.
- Identität: numerische ID reicht nicht zwangsläufig über Plattformen/Quellen; reale Spiegel von verschiedenen Providern mit gleicher Nummer unterscheiden. Bis Composite-Identität muss die Kollision sichtbar und ohne Mischbestand behandelt werden.
- Reports: URL-/Pfadsicherheit, eindeutige Namen, Größe, SHA, PDF-MIME, Binärintegrität, entfernte Reports und Wiederholung ohne Doppelbestand.

Berichte die Differenz zwischen implementiertem, dokumentiertem und benötigtem Vertrag. Bevorzuge einen expliziten Semantikvertrag und gemeinsame Contract-Fixtures vor einer gemeinsamen Bibliothek, falls eine Bibliothek aktuell mehr Migrationsrisiko als Nutzen schafft.

9. SignalKiScanner: mathematische und fachliche Tiefenprüfung

9.1 Parser und Datenqualität

Prüfe parser.py, Referenzparser und echte Roh-CSVs. MT5-Positionsformat hat keinen direkten SL/TP-Beleg; MT4-Orderbuch hat eigene Spalten und Abschlussartefakte. Summenzeile mit Symbol profit und reine Zeitstempelzeile müssen korrekt eingeordnet werden. Inhalt ohne Typ darf nicht unbemerkt verschwinden.

Verfolge Profit, Commission, Swap und Netto mit Vorzeichen, Reihenfolge und Rundung. Prüfe gleichzeitige Trades, gleiche Zeitstempel, Teilpositionen, offene/geschlossene Daten, fehlende Preise, Zeitzonen, ungültige Zahlen und Encoding. Jeder entfernte oder abgewiesene Trade braucht zählbaren Grund.

Identische Zeilen ohne Ticket sind kein Duplikatbeweis. Prüfe aktuelle beweisbasierte Regel mit Rohzahl, Plattform-Positionsanzahl, exakt passendem Snapshot und tatsächlich entfernten Zeilen. Dealzahl versus

Positionszahl dürfen nicht als gleichwertiger Beweis dienen. Trage dieselbe Entscheidung durch Engine, Studie, Rendite, Portfolio, Berichte, Persistenz und KI-JSON.

9.2 Kapitalbasis und Kapitalflüsse

Prüfe `forensics/drawdown.py`, `_implizite_kapitalbasis`, virtuelle Fallbacks und Quellen-Metrics. Unterscheide echte Einzahlungen/Entnahmen, rückgerechnete implizite Basis, virtuelle 10k-Annahme, Anfangskapital der Handelskurve und tatsächlichen Kontostand.

Validiere Priorität der Basisquellen und Verhalten bei 0, negativer, unbekannter oder nicht endlicher Basis. Web-Balance minus kumuliertem Netto ist nur unter passenden Vollständigkeits-/Zeitannahmen belastbar. Teste Cashflows vor Handelsbeginn, zwischen Trades, bei offenen Trades und nach dem Exportfenster. Der Cent-Abgleich muss zur realen Konto-Balance passen und darf virtuelle Fallbacks nicht als echte Einzahlung ausweisen.

Prüfe Trading-Kurve, Originalkonto, kapitalflussneutralen Index und Kopier-Simulation getrennt. Gleiche USD-Schwankungen bei verschiedenen Kontogrößen ergeben verschiedene Prozentwerte. Daraus folgt allein weder Betrug noch Datenfehler. Nenne jede Basis-/Zeitraumabweichung im Bericht.

9.3 Drawdown-Arten und harte Schranke

Prüfe `scoring.dd_maximum`, alle Aufrufer, `ampel_for`, Matrix, `refresh_report_verdict`, DB-Restore, REST und Sync. Die Risikoschranke verwendet das konservative Maximum der vorgesehenen fünf DD-Kanäle: Plattform-Equity, Plattform-Balance, Trading-DD, belastbare Kursreko und Monitorwert. Default ist 30 %, die aktuelle Konfiguration muss offengelegt werden.

Prüfe Peak-to-Trough-DD, Prozent-/USD-DD, relative aktuelle Basis, Intrabarverluste, Fensterbeginn und historische Vollständigkeit. 30,0001 % darf bei Grenze 30 nicht durch Anzeige-Rundung passieren. Gute Rendite oder RetDD dürfen einen belegten Schrankenbruch nie neutralisieren. Fehlende Daten sind unbekannt und nicht automatisch 0 % Risiko.

Der **RetDD-Nenner ist eine andere Auswahl** als das konservative Schrankenmaximum. Es ist der positive belegte Max-Equity-DD einschließlich Floating, nicht beliebiger Balance-/Trading-/Plattform-DD. Prüfe, ob alle verwendeten Monitorwerte dieses Belegkriterium tatsächlich erfüllen.

9.4 Rendite, RetDD und Score

Prüfe `portfolio_statistik.effizienz_kennzahlen`, `ScanResult.refresh_efficiency`, Producers/Consumers, Snapshot und `results_from_db`. Geometrisches Monatsmittel kommt aus ungerundetem End/Start und Dauer vom ersten Open bis zum letzten Close. Ein Jahr hat 365,2425 Tage; ein Monat ist Jahr/12. Leermonate gehören zur Laufzeit.

$\text{RetDD/Monat} = \text{geometrische Monatsrendite} / \text{belegten positiven maximalen Equity-DD}$;
 $\text{RetDD/Jahr} = \text{Jahres-CAGR} / \text{denselben Nenner}$. Der Jahreswert ist nicht einfach zwölfmal der Monatswert. Bei Kapitalfluss-/Fensterabweichungen ist die Kennzahl keine belegte Effizienz des Originalkontos. Kennzeichnung und Auswahlwirkung prüfen.

Aktuelle Regel: geometrische Rendite mindestens konfigurierte Monatsschwelle, Default 5 %, und RetDD/Monat mindestens 1,0; harte Risikoregeln bleiben vorgeschaltet. Das frühere versteckte Score-Gate wurde am 04.10. entfernt. Der Score bleibt Informations-/Matrixwert. Prüfe, ob alte Berichte, Restore, Agenten oder Konsumenten das Gate wieder einführen. Fehlender Equity-DD, $\text{DD} = 0$ oder veraltete Forensik liefern keinen gültigen RetDD/Grün-Weg; messbare Rendite darf trotzdem sichtbar bleiben.

9.5 Equity-Rekonstruktion, GMT und Kurslücken

Prüfe `forensics/equity_rekonstruktion.py`, `equity_studie.py`, `kursdaten.py`, FX und Symbol-Mapping. H1-Bar-Close gilt am Bar-Ende. Kein Look-ahead durch Verwendung eines noch nicht abgeschlossenen Bars. Realisiertes Netto aller exportierten Trades bleibt enthalten, auch wenn deren Floating-Kurse fehlen.

Prüfe Kursabdeckung mit offen gelegtem Nenner: aktive Handelsstunden, Marktpausen, Wochenenden, Holidays, fehlende Bars und unklare Trades. Die Freigabegrenze 95 % wird mit ungerundeten Werten getestet. Eine Anzeige „95 % < 95 %“ muss ihre tatsächliche Präzision oder Ursache zeigen.

Aktuelle GMT-Regel: drei unabhängige Preisereignisse können einen lokalen eindeutigen Shift bei hinreichender Trefferquote belegen; Open/Close-Ereignisse deduplizieren. Dünne/mehrdeutige Wochen können den letzten bekannten Shift erben. Belegter, geerbter und globaler Shift bleiben unterscheidbar. Diese Regel ist eine Modellentscheidung, kein allgemeiner Beweis der Broker-Zeitzone.

Teste DST-Grenzen, geerbte Startwoche, Symbolunterschiede, 4-Sekunden-Scalps, Spread-/Slippage-Ausreißer, extreme Fehlpreise, offene Positionen über Shiftwechsel und nicht-monotone Zeitabbildung. Prüfe harte Unzuverlässigkeitsflags und lokale Lücken. Kleine Preisabweichung darf nicht unbegründet die ganze Woche verlieren; ererbter Shift darf echte Unsicherheit nicht verstecken.

Die Studie und Produktivforensik müssen ihre Unterschiede offenlegen. Aktuelle offene Positionen, Exportende, Referenzbroker und H1-Auflösung begrenzen beide. Eine einzelne zufällige Nähe zum Plattform-DD ist keine Methodenkalibrierung. Nenne fehlende Symbole namentlich und prüfe Weitergabe an GUI, KI-JSON und PDF.

9.6 Martingale, Exposure und Stop-Nachweis

Prüfe die Pflichtbatterie: Martingale-Signatur, Peak-Exposure, Verlustdistanz-/SL-Clustering und DD-Rekonstruktion/Datenabgleich. Prüfe Prüfstatus statt bloßer Anwesenheit eines nichtleeren Dicts. Ein fachlich gescheiterter Test darf nicht als bestanden gelten.

Martingale: Loss-Sequenzen, Lot-Verhältnis, Medianregel > 1,3, Symbolwechsel, Kapitalaufstockung, kleine Stichprobe, Grid-Gruppen und zeitgleiche Positionsschließungen. Trenne Signatur von Strategieabsicht und statistischer Sicherheit.

Exposure: maximale gleichzeitige Anzahl und maximale Netto-/Brutto-/Schock-Exposition müssen nicht zum selben Zeitpunkt auftreten. Gegenläufige Positionen, Multi-Symbol-Netting, Lotwechsel und fehlende Kontrakte prüfen. XAUUSD: 1 Lot bedeutet 100 USD je 1 USD Preisbewegung; 2,66 Lots und 50 USD adverser Bewegung ergeben 13.300 USD, vor Kosten und unter linearer Vertragsannahme. Andere Kontrakte nur aus belegten broker-/instrumentbezogenen Spezifikationen.

SL: fehlender Nachweis bleibt neutral gemäß aktueller Nutzerregel. Prüfe, ob irgendein Score-, Matrix-, Ampel-, Ausschluss- oder Promptpfad trotzdem einen direkten Malus daraus macht. Belegter SL entlastet; statistische Muster sind nicht automatisch Broker-SL. KI darf eine begründete Einschätzung „wahrscheinlich ohne Schutz“ formulieren, muss sie von Beweis unterscheiden. Shock-PnL ist ein Szenario, kein automatisch realisierter Verlust.

9.7 Persistenz, Versionen und Berichte

Prüfe `analysis_version.py`, `db.py`, Forensik-/stats_json, Trade-Snapshots, SHA und Report-Basis. Bestätige die aktuelle Forensik-Version; bei der Bestandsaufnahme war Version 11 maßgeblich. Teste veraltete/neue/inkonsistente Versionskombinationen, geänderte Kriterien, Promptversion, Trade-SHA, Metrics und Kapitalbasis.

Ein alter grüner Bericht darf nicht neben einer aktuell roten oder unbelegten Ampel als aktuelle Empfehlung erscheinen. Kein neuer Forensik-Zeitstempel nur durch DB-Lesen oder Cache-Fallback. Prüfe atomare Speicherung, Transaktionsgrenzen, PDF-Materialisierung, fehlende Fonts, ungültige PDF, Berichtgrößen und Fehlerfortsetzung.

Ampelchronik muss tatsächlich persistierte Läufe und gekippte Einzelkriterien mit exakter Grundlage abbilden. Fehlgeschlagene Prüfung darf kein künstliches Weiß-Flackern oder falschen Wechsel erzeugen. Prüfe gleiche Zeitstempel, Reihenfolge, Wiederholung und Idempotenz.

10. SignalKiScanner: Abläufe, Agenten und Benutzeroberfläche

Rekonstruiere Full-Scan, Teilscan, Quellen-only, MQL-only, Mischmodus, Fix-IDs, manuellen Re-Scan und headless Scan. GUI und autonome Pipeline müssen identische Auswahl- und Bewertungsregeln verwenden. Fix-IDs umgehen nur definierte Vorfilter und Exportauswahl, niemals Risikoregeln. Bei Teilscan müssen relevante gespeicherte Signale auch ohne aktuelle Kataloglistung korrekt eingeordnet werden.

Prüfe agenten/scheduler.py, tageskette.py, scan_launcher.py, Daemon, Lock, Journal und Rollen. Kalender: Werktage, Sonntag, erster Werktag des Monats, Zeitzone Europe/Berlin, DST, verpasste Takte, Rechnerneustart und laufender Job. Marker dürfen nicht vor belastbarem Abschluss fälschlich Erfolg signalisieren. Wiederholung nach Crash darf weder doppelt senden noch still ausfallen.

Lock: PID-Recycling, Prozessstartzeit, veraltete Sperre, zwei Prozesse, Thread-Ausnahme, manueller GUI-Lauf versus Daemon und Bestandsleser. Prüfe Heartbeat-Freshness während langer Jobs, Budgetreservierung, Timeout, Cancel, Freigabe in finally und Journal bei Skip/Fehler.

Betreuer: Quellenfähigkeit, unveränderte Trade-SHA versus neue Metrics/Profilversion, keine erfundenen Dossiers, KONFORM/AUFFAELLIG/STILBRUCH und Idempotenz. Melder: Ampelwechsel, Tagesdigest, Sofortalert, Quellenverweise, doppelte Meldungen und Prioritäten. Chef: Kontextfenster, laufender Scan und Empfehlungen ohne eigenständige Neubewertung.

Streamlit: nutze die installierte Version und passende Skill-/Referenzdokumentation. Prüfe Rerun, st.fragment, Reload/F5, Session-State, Prozess-State, Widgetkeys, Doppelklicks und Hintergrundthreads. Kein Streamlit-UI-Aufruf aus einem ungeeigneten Worker. Unterschiedliche Browser-Sessions dürfen keinen parallelen Produktionslauf starten. Reload darf die Tageskette nicht still abbrechen.

MT5-/Browser-Lifecycle: ausschließlich erlaubte lesende Marktdatenfunktionen, pfadgenaue Prozessverantwortung, portable Startregel, Beenden nach Lauf, Fehler im Cleanup und Konkurrenz zwischen Studie/Scan/Markt. Fremde Terminals bleiben unberührt. Teste Lifecycle mit Mocks, nicht durch unkontrollierten Terminalstart.

LLM: Code rechnet, Modell interpretiert; Ausnahme manuelle Tiefenanalyse nur mit Annahmen und ohne Einfluss auf Ampel/Score. Prüfe alle Promptdateien gegen Defaults, vollständig typisierte Payloads, Herkunft/Einheiten, n/Tradezahl, fehlende Kurse, aktuelle Kriterien, Kostenbudget, Retries, Antwortvalidierung und Cache. Externe Beschreibungen sind Daten, keine Anweisungen. Kein API-/Sessionsecret im Modellkontext oder vollständigen Promptjournal.

REST 8611 und Tradeserver-Sync: gespeicherte Ergebnisse mit aktuellen Kriterien ableiten, ohne Scan/LLM auszulösen; „liest nur“ und „berechnet aktuelle Ampel“ präzise unterscheiden. Prüfe Freshness, unbekannte Filter, Auth, Snapshotkonsistenz, Register/Table/Documents/Complete, Teilfehler, Wiederholung, SHA-Diff, 8-MB-Dateigrenze und Base64-Aufblähung. Alle Kennzahlenfelder und Quelle müssen fachlich konsistent übertragen werden; eigene Rendite-/RetDD-Rohwerte dürfen nicht vorgerundet werden.

11. MqlDownloader: vollständiger Projektprüfplan

Abruf und Selenium: Verfolge Login, Cookie-/Profilhaltung, Detailseite, Export, Sessionablauf und erneuten Login. MT4-History versus MT5-Positionspfad, 404, Login-HTML, DOM-Änderung, lokalisierte Zahlformate und leere Tradehistorie prüfen. Rate-Limit und Cache müssen tatsächliche Requests begrenzen, auch bei Retry und paralleler UI-Aktion.

Kennzahlen und Filter: MPDD-, Monatsrendite-, Wochen-/Abonnentenfilter gegen Code und UI prüfen. Welche Signale werden bereits am Downloader verworfen und sind für den Scanner dann unsichtbar? Wer bewahrt den Ausschlussgrund auf? Monatsrendite nur aus passenden Datumsfenstern; unvollständiger Zeitraum darf keine plausible 0 oder künstliche Rendite erzeugen.

Datenbank und Migration: H2-Schema, Connection-Lifecycle, SQL, Versionierung, Backup, atomare Updates, Blob-/Dateiverweise und Verhalten bei defektem Download prüfen. Keine unvollständige neue Historie über eine intakte alte schreiben. Datenbankmigration mit Kopie alter Schemafixtures testen.

REST: Sortierung/Pagination, MQL4/MQL5-Mehrfachversionen, Trade-/Positionszahlen, S/L-Orderbuch, Initial-Deposit-Felder, Renditedefinition und gespeicherte Reports prüfen. Ein Detailabruf darf keine unkontrollierten Plattformrequests im Serverthread auslösen. Unterschiedliche API-/Dateikonfiguration zu den JavaFX-Monitoren ausdrücklich erfassen.

Swing und Jobs: EDT-Regeln, Hintergrundworker, Cancel, Serienfehler, Fortschritt, Selection-Wechsel und UI-Fehlerzustände prüfen. Eine Statusmeldung darf nicht nur den letzten Teilerfolg zeigen und vorherige Fehler verschweigen. Browser und Worker beim Schließen korrekt aufräumen.

Build, Start und Verteilung: Maven-POM, JDK-Annahmen, Start-/Deployskripte, Pfade mit Leerzeichen, JAR-Ressourcen, Versionierung und Installationswurzel erfassen. Nicht allein im Git-logs/ suchen: durch Code konfigurierte Laufzeitlogs lagen bei der Bestandsaufnahme auch unter C:/Forex/MqlAnalyzer/logs/.

Pflichttests: MT4 und MT5 zum selben Signal, Login-HTML statt CSV, Datumsbereich fehlt, Dezimalkomma, paginierter REST-Katalog, MPDD-Grenze, Sessionretry, Downloadabbruch, beschädigte letzte Datei und Scanner-Akzeptanz aus realem Export.

12. PelicanTrading: vollständiger Projektprüfplan

Zwei-Phasen-Sync: Vollständige UUID-Discovery und nachfolgender Closed-History-Abruf getrennt prüfen. Grenzen von „nur kopierte Provider“, paging, Teilfehler, geänderte Kopierstatus, Provider nicht mehr im Katalog und Resume. Belege, wann die 24h-Frischemarke gesetzt wird; Teilsync darf nicht automatisch „vollständig frisch“ bedeuten.

Tradeabbildung: Provider-/Konto-/Strategie-ID, Zeitstempel, Volumen, Ein-/Ausstieg, Netto-PnL, Gebühren, Swap, gleiche Trades, offene und geschlossene Positionen prüfen. ISO-Zeiten, Server-Zeitzone und Scanner-GMT dürfen nicht doppelt korrigiert werden.

Drawdown: `Trace ReportService.tradeDdAusTrades`, `EquityKurve`, `berechneTradeDd`, `data/trade_dd.csv` und REST. Prüfe geschlossene Kurve plus aktuelles Floating gegen historische Zwischenverluste. Basis aus PnL/Plattform-Inception-Rendite: gleiche Zeitraum-/Währungs-/Gebührenannahmen? Fälle `Yield = 0`, negative Yield, Einzahlungen und unvollständige Historie separat.

Währungen: USD, USC und übrige Währungen einschließlich offline Cache. `FxRates` und REST-Konvertierung anhand eingehender Originalbeträge und ausgehender USD prüfen. Bei einem Referenzkurs für alle historischen Geldwerte die Methode als Snapshotumrechnung kennzeichnen; sie ist nicht automatisch Umrechnung zum jeweiligen Tradezeitpunkt. Preis, Lot, Prozent und Geldbetrag dürfen

nicht alle mit demselben Faktor skaliert werden. DD-Prozente nur dann invariant, wenn Zähler und Basis konsistent gleich skaliert werden.

Strategiemodul: Erfasse anhand tatsächlicher Call-Graphen sämtliche Funktionen in/um `strategien.db`. Falls der Code Handels-/Kopieraktionen anbietet, prüfe Autorisierung, Auswahl, Parameter, Demo/Live-Verwechslung, Wiederholung und Schutz vor Doppelklick ausschließlich statisch oder mit Fake-Transport. Keine reale Aktion auslösen.

Store und UI: `geladen.json`, Trade-Datei, DD-Datei, Abonnenten-DB, UI-Prefs und Migration als zusammengehörigen Snapshot betrachten. Cache-First darf nicht allein aus einem lokalen Datei-Hash auf unveränderte Remote-Trades schließen. Freshness separat nachweisen. Charts, Badge und Reports müssen dieselbe Basis erklären.

LLM: lokale Risikoanalyse und Scanneranalyse haben unterschiedliche Aufgaben. Prüfe Score 1 bis 10, fehlendes SL, Statistikherkunft, Promptränder, Antwortparser, Berichtsspeicherung und echte /reports-Unterstützung am aktuellen Code. Historische Aussage „Reports leer“ nicht ungeprüft übernehmen.

Pflichttests: phase-1-only Erfolg, Phase 2 abgebrochen, Providerwechsel, USC-Skalierung, EUR offline mit/ohne Cache, Währungswechsel, historischer Floating-Crash trotz positiver Closings, DD-Datei-Update und Scanneraufnahme desselben Snapshots.

13. PelicanWinnerLooser: eigener fachlicher Prüfplan

Dieses Projekt untersucht Kopierkonten, nicht bloß Providerqualität. Erstelle eine eigene Architektur-/Datenflusskarte für API, Session, Crawler, Arbeitsplan, Account-/Provider-Repositories, Analytics und JavaFX-Ansichten.

API und Identität: Profile, Account-/Provider-/Copier-IDs, per Konto mehrfach sichtbare Beziehungen, paginierte Liste, 404 und verbotene Datenfelder prüfen. Profile-404 kann dokumentierte Capability-Grenze sein; zwischen „nicht vorhanden“, „nicht zugänglich“, Sessionfehler und transientem Fehler unterscheiden.

Resume und Arbeitsplan: Prüfe Statusübergänge pending/running/done/failed/paused im tatsächlichen Schema. Nach Sessionablauf, Prozesscrash, Abbruch und Netzfehler muss der Arbeitsplan vollständig und ohne Double-Counting fortsetzbar sein. Ein erledigter Teil darf nicht verloren gehen; ein nicht erledigter Teil darf nicht als done markiert werden. Prüfe neue Konten bei Wiederaufnahme und Auswahländerungen.

Auth: DPAPI-Speicher, SessionService, LoginManager, SessionWiederaufnahme und manuelle Sessionoptionen prüfen. Konto-/Benutzerbindung, beschädigtes Ciphertext, fehlende DPAPI, Datei-Rechte, Temp-Dateien und Logging. Ein maskiertes GUI-Feld beweist keine sichere Speicherung. Kein Sessionmaterial in Evidenz/PDF.

Rate-Limit und Abbruch: Retry-After, Pausegrund, Retrybudget, endlose Arbeitspläne, UI-Abbruch, Threadinterrupt und fairer Fortschritt. RateLimiter mit gefrorener Uhr und Fake-Transport testen. Wiederaufnahme soll den Fehlergrund beseitigt haben und nicht nur denselben Fehler erneut zählen.

Stichprobe: Wer ist überhaupt beobachtbar? Aktive/geschlossene Kopierkonten, Survivorship, fehlende/gesperrte Profile, zeitliche Verschiebung, Mehrfachkonten pro Person, unterschiedlich lange Historie und wiederholte Snapshots. Zeige Grundgesamtheit, erreichbare Menge, ausgewertete Menge und Ausschlüsse. Ergebnisse nicht als repräsentativ für alle Anleger darstellen, wenn die API diese Auswahl nicht belegt.

Statistik: Netto-/Bruttogewinn, Startkapital, Ein-/Auszahlungen, Währung, Zeitraum, Quantildefinition, Median, Top-10-%-Konzentration, negative Gesamtsumme und $n = 0/1/\text{klein}$ prüfen. Geldgewichtete und kontogewichtete Verteilung unterscheiden. Gewinneranteil nicht mit erwartbarer Anlegergewinnchance

verwechseln. Unsicherheit nur mit passenden Annahmen und unabhängig berechneten Verfahren ausweisen.

Kurven/Overlay: Gleicher Startpunkt, Prozentbasis, Cashflows, Zeitraster, Nullwerte, Lücken, Forward-Fill, Währungsumrechnung und Konten mit unterschiedlichen Starttagen prüfen. Ein schöner Overlay darf unbekannte Abschnitte nicht als gemessene Kurve darstellen. Ranglisten müssen Sortierung, Filter, missing-last und Denominator erklären.

FX/EUR: Originalbetrag erhalten, Kursdatum/Quelle und Offline-Fallback sichtbar. Heute umgerechneter historischer Gesamtgewinn ist keine historische EUR-Rendite ohne weitere Berechnung. Bei stückweiser Umrechnung identische Zeit-/Einheitsregeln für Konto und Kurve verwenden.

Pflichttests: Ablauf mit 401 mitten im Paging, Profile-404, Crash nach DB-Commit vor Workitem-Commit, Resume nach Settingsänderung, leerer Bestand, ein Konto, mehrere identische Gewinne, Verluste mit negativer Gesamtbasis, mehrere Währungen, Cashflows, Kurvenlücken und unabhängige Quantil-Nachrechnung.

14. RoboForex: vollständiger Projektprüfplan

MT4/MT5-Unterschied: MT4-Positionen und MT5-Rohdeals getrennt halten. TradeAggregator muss IN, OUT, INOUT, Teil-Schließung, mehrere Öffnungen/Schließungen, Restvolumen, Hedging/Netting, Reihenfolge und gleiche Zeitstempel abbilden. Offene Reste nicht als geschlossene Position exportieren.

PnL-Erhaltung: Summe zugeordneter Profit/Commission/Swap muss mit Rohdeals stimmen. Gebührenallokation bei Teilclose, separate Kosten-/Balanceevents, ungültige Lots und Preisfelder testen. Keine doppelte Commission, kein Verlust von unzuordenbaren Deals ohne Audit. Jede Aggregation muss prüfbare Herkunfts-IDs erhalten.

Zeitlicher Positionsverlauf: Der gelesene Aggregator verdichtet alle Deals einer Positions-ID zu einem Trade vom ersten IN bis zum letzten OUT. Summenerhaltung beweist damit noch keine richtige zeitliche Exposure-/Equity-/Realisierungskurve. Die offene-Positionen-Zählung verwendet im untersuchten Pfad Ereigniszahlen statt Restvolumen. Teste ausdrücklich ein IN über 1 Lot und ein OUT über 0,25 Lot: 0,75 Lot bleiben offen, auch wenn IN- und OUT-Zahl gleich sind. Verfolge die tatsächliche Weiterverwendung, bevor du daraus einen vollständigen Produktbefund machst. OUT ohne IN wird in beschnittener Historie mit angenähertem Open gerettet; diese synthetische Herkunft muss in Verlustdistanz, Haltedauer und H1-Rekonstruktion sichtbar bleiben.

CSV-Nettovertrag: Robo exportiert im gelesenen REST-Pfad bereits Netto-PnL in Profit und zusätzlich echte Commission/Swap-Spalten. Vantage exportiert `closeNetPnL` mit enthaltenen Gebühren und neutralen zusätzlichen Kostenfeldern. Prüfe pro Quelle die genaue Parsersemantik mit Rohantwort, Cache und CSV; ein allgemeines erneutes Addieren der Kosten kann falsch sein. Leite keinen Fehler allein aus Spaltennamen ab.

Capability: Plattform `rst` oder unbekannter MT-Wert darf nicht im MT4-/MT5-Pfad scheinbar erfolgreich laufen. Logwarnung, UI-Status, REST-Verfügbarkeit und Scanner-Folge miteinander prüfen. Nicht automatisch als Netzwerkfehler behandeln.

Drawdown und Renditebasis: Closed-PnL-Kurve, Yield-/10k-Rückrechnung, DD-CSV und REST-Metrics prüfen. Keine Gleichsetzung mit historischem Equity-DD. Der aktuelle Hersteller-/Broker-DD bleibt anderer Kanal. Datenfenster, Anfangskapital und gefilterte Deals offenlegen.

Stores und Nebenläufigkeit: DealStore, `geladen.json`, SubscriberDb, Sessionmigration und DD-Cache auf atomic replace, gleichzeitigen Download/REST/UI, Crash und Dateisystem-Sperren prüfen. 7/30-Tage-Snapshots: Zeitgrenze, mehrere Snapshots pro Tag, Rückgang, Löschung und fehlende Basis.

Reports/UI: offene Positionen, Trades, Tages-PnL, Export, Signalwechsel, Broker-/KI-Risiko und Badge prüfen. Robo-Systemprompt als Referenz lesen, aber seine tatsächliche Injection-/Antwortabsicherung testen. Serienfehlerlimit und 24h-Frische dürfen nicht nur Tooltipregeln sein.

Pflichttests: ein IN und zwei OUT, mehrere IN vor OUT, Gebühren auf Entry und Exit, Reversal, offene Restmenge, ungeordnete Deals, `rst`, Dateiaustausch während REST, Subscriber-Lücke und großem historischen Floating bei kleinem Closing-DD.

15. Vantage: vollständiger Projektprüfplan

Katalog und Kategorien: Kategorien/Paging, Parameterschema, Rankingwechsel und Grenzen der Abrufmenge prüfen. Verfolge wiederkehrendes `10402 err_invalid_param` aus Logs bis Requestbuilder und UI. Ist es falscher Parameter, fehlende Capability, abweichender Endpunkt oder Anbieteränderung? Fehlergrund und Vollständigkeit müssen für Nutzer/Hub sichtbar sein.

Tradehistorie: Dealgenaue Positionen, 20.000-Trade-Grenze, paginierte Downloads, Zeitfenster, Countangaben, abgeschnittene Exporthistorie und doppelte Seiten prüfen. Endliche Max-Anzahl darf nicht als gesamte Laufzeit beschrieben werden. Alter aus lokaler Historie ist Untergrenze, wenn ältere Trades fehlen.

Währung und Instrument: USD versus USC, Drittwährungs-404 mit nachvollziehbarem Grund, `currencyCode`, Basis-Symbole versus `.sc`-Suffix, Commission im Netto-PnL. `AumUsd` ist Kopierer-Kapital und darf nicht als Provider-Balance/Initial Deposit dienen. REST, GUI und LLM müssen das gleich behandeln.

Zeit-/Python-Vertrag: Java und `download_trades.py` konvertieren im gelesenen Pfad Epoch-Zeiten in die Systemzeitzone. Prüfe denselben Rohtrade auf zwei Rechner-Zeitzone sowie vor/nach DST. Zeitzoneinformation darf beim CSV-/Cache-Export nicht zu einer scheinbar belegten Brokerzeit werden; eine zusätzliche Scanner-GMT-Korrektur kann einen bereits konvertierten Zeitstempel erneut verschieben. Rohzeit, Umrechnung, Zeitzone und Bedeutung der Ausgabe müssen nachvollziehbar sein.

Rendite und DD: gemessene 30-Tage-Rendite gegen hergeleitete Laufzeit-/Yield-Rendite trennen. Closed-PnL-DD und angenommene Basis nicht in belegten Equity-DD umbenennen. Prüfe historische Floating-/Open-Positionen-Abdeckung.

Historie/Store: `TradeListStore`, geladene Hashes, `CopierDb`-Migration, tägliche Snapshots, Frische und bei Filterwechsel sichtbare Batchmenge prüfen. Eine unveränderte lokale CSV sagt nichts darüber, ob der Anbieter inzwischen neue Trades hat. Cache-Freshness verlangt Updateprüfung oder ehrlich ausgewiesenes Offlinealter.

Reports und JavaFX: Broker-Risiko-Klassifizierung, KI-Score, Export, Tages-PnL, offene Trades und Ladezustände prüfen. Numerisches Rating mit Text-/fehlenden Kategorien und geänderten Brokerwerten testen. Nicht alle Signale pauschal in dieselbe Risikoklasse fallen lassen.

Pflichttests: `invalid-param` im Katalog, zwei gleiche Paginationseiten, mehr als 20.000 Trades, USD/USC/Drittwährung, Meta-Symbolsuffix, Profit mit Commission, entfernte Provider, Katalog frisch/Trades alt und Scanneranalyse ohne reale Kapitalbasis.

16. ZuluMonitor: vollständiger Projektprüfplan

Dual-Host-Fallback: HOSTLINE und Alternativhost, fehlerhafte JSON-/Tradeantwort, Seite 0/folgende Seiten, Timeout und vollständiger Hostwechsel prüfen. Kein Mischen verschiedener Snapshots ohne Nachweis. Fallback nur bei passenden Fehlern; 401, 404, 429 und fachlicher Fehler brauchen getrennte Behandlung. Retry muss begrenzt sein.

Verifizierter Tradebestand: .meta-Marker, SHA-/Zähl-/Zeitbezug, geladen.json, Restarts und Offlineexport prüfen. Marker erst nach vollständig erfolgreichem Download. Marker ohne Datei oder Datei mit falschem Hash gilt nicht als verifiziert. Abgebrochener neuer Download darf den letzten intakten Snapshot nicht vergiften.

Währung und Kontotyp: PnL in Trader-Kontowährung; USD-Freigabe der Scanner-CSV anhand aktuellen Codes bestätigen. FX-Symbole ohne Schrägstrich, Netto-PnL, Demo flag, Real/Demo-Klassifizierung und unbekannte Kontowährung testen. Demo darf nicht still als Livebeweis dargestellt werden.

Rendite/DD: ROI/Laufzeit-Näherung, geschlossenes Trade-PnL, virtuelle/rückgerechnete Basis und historische Floating-Lücke prüfen. ROI/Laufzeit ist keine geometrisch gemessene Monatsrendite. GUI-/REST-Feldnamen und Hub-Denominator kritisch abgleichen.

Abonnementhistorie: README enthält alte Aussage ohne Historie und neuere SubscriberDb-/history-Beschreibung. Bestimme den aktuellen Vertrag aus Code und Tests. 7/30-Tage-Änderungen bei zu kurzer Historie müssen unbekannt bleiben; aktueller Abonnentenstand ist keine Verlaufskurve.

LLM und UI: leerer Systemtext im gelesenen ReportService-Aufruf, externe Beschreibung, Risikoregeln, Antwortschema und tatsächliche Datenabgrenzung prüfen. Führe lokale adversarielle Payloadtests ohne Modellkosten aus; echter Modelltest bleibt separate Liveprüfung. Trades/Open-Positionen/Export-Cache/Frischeanzeigen wie bei anderen Monitoren prüfen.

Pflichttests: erster Host ungültig, zweiter gültig; erster Host Teilpaging; beide ungültig; Meta-Datei fehlt; alter Meta-SHA; Nicht-USD-Konto; Demo-Trader; 6 Tage Subscriberhistorie; Provider weg; Prompttext enthält falsche „Ignoriere Regeln“-Anweisung.

17. Logs: Inventur, Korrelation und Root-Cause-Verfahren

Suche nicht nur nach *.log. Erfasse rotierte Logs, gzip/zip, stdout/stderr, Java-Stacktraces, Python-Tracebacks, Surefire-XML/TXT, Journal-/Jobtabellen, Scanprotokolle, Error-Marker, Browser-/Crawlerdiagnostik und Startskriptaushaben. Ermittle konfigurierte Laufzeitpfade aus Logger-/Configcode und aktuellen Startparametern.

Ausgangspunkte aus der Bestandsaufnahme: Scanner data/scan_workflow.log, data/agenten_daemon.log, data/streamlit_restart.log und Agententabellen; Robo data/robomonitor.log; Vantage data/vantagemonitor.log; Zulu data/zulumonitor.log; PelicanTrading data/pelicanmonitor.log; WinnerLooser data/pelican-winner-looser.log; MqlDownloader unter anderem C:/Forex/MqlAnalyzer/logs/application.log, rotierte Application-Logs und Konvertierungs-/Downloadprotokolle. Mögliche abweichende Installationspfade neu aus dem Logger bestimmen.

Arbeite in fünf Schritten:

1. Manifest aller relevanten Dateien/Tabellen mit Größe, Zeitraum, Format und zugänglichem Anteil. Produktions-DB nur mit nachgewiesenen Read-only-Verbindungen oder konsistenter Sicherung öffnen; keine Initialisierung/Migration zum „Lesen“ aufrufen.
2. Maschinell Ereignisse extrahieren und normalisieren: error/warn/traceback, HTTP-Fehler, Timeout, Retry, Parse-/Schemafehler, FX/Kurslücke, Lock/Heartbeat, Abbruch, Stale und Erfolg/Complete. Redigiere Geheimnisse vor Ausgabe.
3. Cluster nach Signatur, Projekt, Quelle, Zeitpunkt, Run/Signal und Codeversion. Häufigkeit, erster/letzter Treffer, Folgefehler und betroffene Datenmenge erfassen. Ein Fehler mit 100 Retries ist nicht automatisch 100 unabhängige Fehler.

4. Für jedes wichtige Cluster vollständigen Ablaufkontext lesen und Codepfad nachvollziehen. Ersten ursächlichen Fehler vom Folgefehler trennen; Logs mit DB-/Datei-/Snapshotstand abgleichen. Erfolgsmarker nach Fehler nur glauben, wenn Resultat und Vollständigkeit belegbar sind.
5. Status: aktuell reproduziert, aktueller Code bestätigt, historisch und behoben, erwartbare Grenze, intermittierend oder ungeklärt. Zu jedem offenen Cluster konkreten Repro-/Monitoring-/Diagnosebedarf nennen.

Prüfe Logqualität selbst: Run-/Korrelations-IDs, strukturierte Felder, Zeitstempel/Zeitzone, Severity, Stacktrace, redigierte URLs, Rotation/Retention, Loggerfehler, mehrfaches Logging und Kosten. Bei fehlender Korrelation eine begründete Verbesserung vorschlagen. Logs sind Belege für Ereignisse, keine alleinigen Belege für die Richtigkeit von Berechnungen.

18. Ablaufrekonstruktion vom Klick bis zum Ergebnis

Erstelle für die folgenden Abläufe eine Sequenz-/Zustandsbeschreibung mit Dateien, Funktionen, Threads, Requests, Persistenz, Erfolgskriterium und Fehlerpfad. Diagramm und Fließtext müssen denselben Ist-Zustand zeigen.

Ablauf	Zu beantwortende Kernfragen
Suite starten	Welche Anwendung wird wirklich gestartet? Welche Prozesse gehören zur Suite? Wann gilt sie als bereit?
Signale laden	Vollständiger oder gefilterter Katalog? Paging korrekt? Was bleibt nach Teilfehler erhalten?
Einzelsignal auswählen	Welche Daten kommen aus Session, Datei, DB oder Remote? Wie wird Alter angezeigt?
Einzel-/Batch-Tradeabruf	Welche Menge bedeutet „sichtbar/alle“? Was passiert bei Cancel, 429 und Serienfehler?
Hub Full-/Teilscan	Wer wählt Kandidaten? Welche Fix-IDs? Was passiert mit offline bekannten Signalen?
Forensik und Studie	Welche Trades, Cashflows, Kurse, Kontrakte, GMT und Messfreigaben?
KI-Bericht und Portfolio	Exakte Zahlenbasis, Prompt, Budget, Antwortprüfung, Persistenz und Stale-Handling?
Agenten-Tageskette	Scheduler, Rollen, Lock, Heartbeat, Restart, Journal und Meldungen?
Tradeserver-/REST-Ausgabe	Snapshot, aktuelle Kriterien, Herkunft, ungerundete Werte und Teilfehler?
WinnerLooser-Kontencrawl	Auswahl, Workplan, Sessionpause, Resume, Stichprobe und Statistik?
Offline-Neustart	Welche cached Daten bleiben belastbar? Wo fehlen Updateprüfung oder echte Freshness?
Schließen/Abbruch	Cleanup von Threads, DB, HTTP, Browser und Terminal; was ist danach wiederaufnehmbar?

Dokumentiere auch Nicht-Erfolg: „API antwortet“, „Port ist offen“, „Datei existiert“ und „Job ist fertig“ sind jeweils andere Aussagen als fachlich vollständiger Erfolg.

19. Unabhängige Rechenorakel und Grenzfallkatalog

Berechne Sollwerte in kleinen unabhängigen Prüfskripten. Importiere nicht die zu prüfende Funktion, um anschließend denselben Algorithmus nur noch einmal zu bestätigen. Nutze klar definierte synthetische Ereignisse und belegte reale Daten. Formeln, Einheiten und Annahmen müssen im Reviewbericht stehen.

ID	Fixture / Eingabe	Unabhängige Erwartung
M01	Start 10.000, End 12.100, exakt 2 Projektmonate	Geometrisch 10 %/Monat; CAGR/Jahr = $100 \times (1,21^6 - 1) = 213,8428376721 \%$
M02	M01 und belegter Equity-DD 5 %	RetDD/Monat = 2; RetDD/Jahr = 42,76856753442; Jahreswert ist nicht 12 x Monatswert
M03	M01, nur Closing-DD 5 %, keine Equity-Messung	Rendite messbar; RetDD unbekannt; kein Grün über RetDD
M04	Belegter Equity-DD = 0	Quotient unbekannt, keine Infinity als Mindestqualität
M05	RetDD = 0,9995 / 1 / 1,0005	Unter 1 bleibt gesperrt; genaue Grenze und Rohwerte prüfen
M06	DD 29,9999 / 30 / 30,0001 bei Limit 30	Grenzvergleich ungerundet; letzter Wert bricht Schranke
M07	Fünf DD-Werte 15/20/12/26/31	Konservatives Maximum 31; guter RetDD hebt Rot nicht auf
M08	Kein DD-Kanal vorhanden	Unbekannt bleibt fachlich unbekannt; keine positive Risikoaussage
M09	10.000 Balance, historisch Floating -4.000, später Closing +100	Echter beobachteter Equity-DD 40 %, Closing-Kurve kann 0 % zeigen
M10	Nur heutiges Floating plus Closings für M09	Heutiger Endpunkt rekonstruiert den früheren Floating-Verlust nicht
M11	Einzahlung 10.000 bei unveränderter Marktposition	Kein künstlicher Profit; Flow-neutraler Index braucht zeitgleiche Bewertung
M12	Auszahlung bei offenen Trades	Keine künstliche Strategiever schlechterung allein durch Kapitalabzug
M13	Cashflow genau an Trade-/Barzeit	Definierte Ereignisreihenfolge, reproduzierbare Equity/Index-Ergebnisse
M14	Zwei gleiche CSV-Zeilen, Plattformzahl deckt Rohzahl	Beide erhalten; identische echte Legs bleiben echt
M15	Exakte Zwillinge und passend belegte bereinigte Positionsanzahl	Nur belegte Doppellieferung entfernen; alle Verbraucher gleich

ID	Fixture / Eingabe	Unabhängige Erwartung
M16	Plattform-Dealzahl statt Positionsanzahl	Kein falscher Dedup-Beweis ohne Format-/Snapshotgleichheit
M17	Netto-Trade ohne Kursdaten	Realisiertes Netto bleibt enthalten; Floating-Lücke sichtbar
M18	H1-Bar beginnt 10:00, endet 11:00	Close erst am Ende verfügbar; keine Vorwegnahme
M19	Abdeckung 94,999 / 95 / 95,001 %	Freigabe ohne Vorrundung; UI präzise genug
M20	Drei Preisereignisse mit eindeutigem Shift	Aktuelle GMT-Regel prüfen; echte Ereignisse statt doppelt gezählte Trades
M21	Dünne Woche nach belegtem Shift; echter DST-Wechsel später	Geerbte Annahme kenntlich; Preisfehler/Lücken und harte Grenzen bleiben
M22	Drei identische/korrelierte Endpunkte	Keine übertriebene unabhängige Stichprobe behaupten
M23	XAUUSD 2,66 Lots, adverser Move 50 USD	13.300 USD lineares Exposure, vor Kosten
M24	USC-PnL 12.345 / USD-PnL 123,45	Gleiche USD-Nettosumme; Preise/Lots unverändert
M25	Gewinnmonate mit mehreren Leermonaten	Zeitspanne vollständig; kein künstlich erhöhtes Monatsmittel
M26	Endkapital ≤ 0 oder Basis ≤ 0	Kein regulärer geometrischer Ertrag/RetDD aus ungültigem Nenner
M27	9, 10, 1 403.03, bool, NaN, Infinity	Definierte Parse-/Ablehnungsregeln, keine stillen falschen Risikowerte
M28	Zwei IN, zwei Teil-OUT mit Kosten	Volumen und Gesamtnetto erhalten; kein Gebührenverlust
M29	Schließen und Öffnen zur selben Sekunde	Definierte Sortierung; kein erfundener Exposure-Peak
M30	WinnerLooser $n = 0/1$, Ties, negative Summe	Definierte Statistik-/Rankingwerte oder ehrliches unbekannt

Ergänze metamorphe Prüfungen: Reihenfolgenvariation darf bei unabhängigen Trades keine PnL-Summe ändern; USD/USC-Skalierung erhält konsistent definierte Prozentgrößen; neue irrelevante Daten dürfen alte belastbare Trades nicht still verwerfen; korrigierter Snapshot muss Version/SHA und abhängige Berichte ändern. Grenzen dieser Invarianten explizit benennen, etwa bei zeitabhängiger Kursbewertung oder tatsächlich revidierten Daten.

20. Projektübergreifende E2E- und Ausfallszenarien

Verwende lokale Fake-REST-Server, Temp-Dateisysteme und Testdatenbanken. Trace jeden Fall mit Producer- und Consumerbeleg. Priorität ist Schutz vor falscher positiver Bewertung und Datenverlust.

ID	Szenario	Verpflichtende Verifikation
E01	MQL4-Orderbuch bis Hub-Bericht	SL-Felder, Netto, Datenqualität, Ampel und PDF konsistent
E02	MQL5-Positionen ohne SL-Spalten	Fehlender Nachweis neutral in allen Bewertungspfaden
E03	Jede der fünf Quellen mit eigenem Export	Realer Quellenvertrag wird vom Scanner korrekt akzeptiert
E04	Quelle liefert Closing-DD unter EQ-Feldname	Keine ungeprüfte Aufwertung zum Floating-Nachweis
E05	Geänderte Metrics bei gleicher Trade-SHA	DD/Kapitalbasis/Kriterien und Berichte korrekt invalidiert
E06	Quelle offline, Cache intakt	Bekannter Stand mit wahren Datenalter; kein erfundener Neuabruf
E07	Quelle offline, Cache-SHA falsch	Belastbarer alter Stand nicht vorgetäuscht; klarer Fehler
E08	Katalog frisch, Trades/Metrics aus alten unterschiedlichen Ständen	Snapshotbruch sichtbar; positive Freigabe fachlich geprüft
E09	Zwei Quellen mit gleicher numerischer ID	Kein Mischbestand/überschriebener Provider; Spiegel ausdrücklich belegen
E10	Version/Kürzel/Reportname mit Pfadsegmenten	Kein Zugriff außerhalb erlaubter Artefaktverzeichnisse
E11	Export ist Login-HTML	Kein CSV-Erfolg, kein Überschreiben des guten Snapshots
E12	HTTP 429/5xx-Serie in Batch	Limit, Pause, Cancel und tatsächliche Teilvollständigkeit
E13	Crash zwischen Datei-, Ledger- und DB-Update	Intakter Snapshot oder sichtbarer recoverbarer Zwischenstand
E14	Gleichzeitiger Download und REST-CSV	Keine halbe Datei oder SHA-/Zählerkombination unterschiedlicher Stände
E15	Full-Scan mit Fix-ID außerhalb Katalog	Definierter Nachladeweg, Format korrekt, normale Bewertung
E16	Teilscan mit grün/gelb plus Fix-IDs, Quelle offline	Scope vollständig; keine still verschwundenen Pflichtsignale

ID	Szenario	Verpflichtende Verifikation
E17	Forensikversion/Prompt/Kriterien ändern	Stale zuverlässig; neue Berichte verwenden aktuelle Basis
E18	Studie und Produktivscan aus demselben Snapshot	Erklärbare Unterschiede; identische Trade-/Cashflow-/Dedup-Grundlage
E19	GUI-Reload/Doppelklick während Tageskette	Ein Lauf, fortlaufender Journalstatus, kein verlorener Digest
E20	Zwei Prozesse plus PID-Recycling	Lock schützt realen Besitzer und lässt abgestürzten Lauf recovern
E21	Monatsjob über DST/Rechnerneustart	Termin und Erfolgsmarker ohne Doppel-/Fehllauf
E22	LLM-Ausfall/Antwort ungültig/Budget leer	Forensik bleibt korrekt, kein erfundener KI-Bericht oder Score
E23	Tradeserver akzeptiert Tabelle, PDF scheitert	Richtiger Teilfehler, Wiederholung/Diff, kein falsches Complete
E24	REST 8611 mit Stale, Filter und Auth	Deterministische Felder, Freshness und kein Netzwerk-/LLM-Nebeneffekt
E25	WinnerLooser Session abläuft nach Teilfortschritt	Pause/Resume mit vollständigem Workplan ohne Double-Counting
E26	Rechner offline neu starten	Cache-Verfügbarkeit plus ehrliche Freshness und Featuregrenzen
E27	Fremde Anwendung belegt Suite-Port	Startskript darf fremde Prozesse nicht ungeprüft beenden
E28	Drittwährung/USC/Demo über GUI und REST	Keine falsche USD-/Live-/Kapitalbasisinterpretation

Prüfe erfolgreiche Wege und absichtlich fehlschlagende Wege. Zähle Testfälle mit passendem Sollwert, nicht nur „Request war 200“. Eine optisch korrekte Tabelle beweist keinen korrekten Denominator.

21. Sicherheit und Vertrauensgrenzen

Prüfe defensive Sicherheit an realen Ein- und Ausgabepunkten. Kein unspezifischer Sicherheitskatalog ohne Codebezug.

- Secrets: Env/DPAPI/SecretStore, Cookies, Token-Dateien, POM/Properties/JSON, Git-Historie soweit begründet, Backups, LLM-Journal und Fehlermeldungen. Report nur redigiert, keine Werte ausschreiben.
- Netzwerk: bind localhost versus LAN, optionales Token, Health-Ausnahme, Header/Query, Redirects, TLS-Validierung, Timeouts und Limits. Token in URL kann in Logs landen; tatsächliches Logging prüfen.
- Dateipfade: Quelle/Kürzel/Version/Reportname, URL-Decoding, Traversal, Kollision sanitierter Namen, absolute Pfade, Windows-spezifische Zeichen und erlaubte Basisverzeichnisse.

- Fremdtext: JSON/HTML/CSV/Strategie-/Reviewtext und LLM-Antwort sind untrusted Daten. Prompt-Injection, HTML-/Markdown-Ausgabe und mögliche Scriptlinks nur an tatsächlichen Renderpfaden prüfen.
- SQL/Datenbank: Parameterbindung, dynamische Spalten/Orderby, erlaubte Werte, Migrationsverantwortung und Read-only-Pfade.
- Prozessaktionen: Pfad/PID/Startzeit als Besitznachweis, breite taskkill-/Portkill-Regeln, Cleanup, Terminal-/Browserprofile und Startskriptargumente.
- Ressourcen: große JSON/CSV/PDF, endloses Paging, Retry-Stürme, Thread-/Connection-Leaks und Speichergrenzen.
- Handlungsfähige Module: Handels-, Kopier-, Deploy- und Sync-Funktionen erfassen und ihre Autorisierung/Idempotenz statisch oder mit Mocks prüfen. Keine realen Aktionen.

Falls ein möglicher Secretfund den Bericht gefährdet, dokumentiere nur Fundort, Art, Zugänglichkeit und Abhilfe. Redigierung gilt auch für Screenshots, Testoutputs und Agentennachrichten. Prüfdaten dürfen keine Passwörter, Sessioncookies oder Accountdaten an externe Dienste schicken.

22. Performance, Wartbarkeit, Build und Betrieb

Performance: Miss in isolierten Datenbeständen kleine/mittlere/große Kataloge und Tradehistorien. Nenne Hardware, Datenumfang, Warm-/Cold-Cache, Laufzeit, Speicher und dominante Codepfade. Prüfe N+1-Requests, wiederholtes CSV-Parsen, Chart-/PDF-Generierung pro Rerun, DB-Indizes, Hashkosten und Serialisierung. Keine Performancebehauptung ohne Messung oder klar belegte Komplexitätsursache.

Nebenläufigkeit: JavaFX Application Thread/Swing EDT, Executor-Lifecycle, Cancel/Interrupt, Python-Threads, Lockreihenfolge, gemeinsam genutzte Clients, Temp-Dateinamen, SQLite-/H2-Verbindungen und Snapshotatomicität. Atomarer Dateiaustausch allein macht Datei plus Ledger plus DB nicht gemeinsam transaktional.

Wartbarkeit: Duplikate zwischen Monitoren, monolithische Appklassen, UI-/HTTP-/Math-Kopplung, harte Konstanten, Exception-Swallowing, unklare Optionalwerte und Namenssemantik prüfen. Refactoring nur mit konkretem Nutzen, Abhängigkeiten, Migrationsplan und Gegenposition begründen. Gemeinsame Vertragstests können zunächst sicherer als Frameworkvereinheitlichung sein.

Build: tatsächlicher JDK/Maven/Python, Dependencyversionen, transitive Bibliotheken, offene Requirements-Ranges, Plugins, Resources/Fonts, Packaging, Wrapper, Offlinebuild und reproduzierbare Umgebung. Veralterte Abhängigkeit ohne konkrete Verwundbarkeit/Funktionsfolge nicht automatisch als P1 deklarieren. Falls aktuelle CVE-/API-/Anbieterinformationen nötig sind, nur passende aktuelle Primärquellen verwenden und Abrufdatum nennen.

Betrieb: Startall-Serviceinventar, Readiness/Health, Portkonflikt, Boot-Reihenfolge, Daemon-Autostart, Statusanzeige, Shutdown, Backup/Restore, Datenmigration, Retention und Upgrade/Rollback. Dokumentiere welche Regeln tatsächlich implementiert sind. Kein pauschaler Vorschlag „alles als Service/Docker“, ohne Windows-/JavaFX-/MT5-Anforderungen zu berücksichtigen.

UI: Prüfe UI_KONVENTIONEN.md gegen jede aktuelle Oberfläche: Toolbar, Ampel, Broker/KI, DD-Badge, Tabs, CSV-Export, Freshness und APIstatus. MqlDownloader ist laut Konvention ausgenommen; WinnerLooser nur Referenz, daher Abweichungen nicht automatisch Fehler. Relevanter als identische Optik sind eindeutige Kennzahlbasis, Vollständigkeit, Fehlerstatus, Cancel, Batchscope und fehlende Daten.

23. Teststrategie und reproduzierbare Ausführung

Ermittle Tests aus dem aktuellen Dateibaum und Build, einschließlich zusätzlicher Testmodule und unversionierter neuer Tests. Prüfe Testqualität: unabhängige Sollwerte, Assertions auf Inhalte statt nur Dateiexistenz/HTTP-Status, deterministische Uhren, temporäre Pfade und Mocks für Netzwerk/Secrets/Trading.

Für den Scanner ist `pytest.ini` mit Standardausschluss der `llm`-Tests vorhanden. Verwende den passenden Projektinterpreter und `PYTHONPATH`, statt einen beliebigen Systeminterpreter. Prüfe Fixtures vor dem Lauf auf echte Datenpfade/Netzwerk/MT5-Nebenwirkungen. Für Java erfasse `POM/Profile/Failsafe/Surefire` vor `mvn test`; kein `run.bat`, `start.bat`, `deploy` oder `GUIstart` als Testabkürzung.

Ein Ausführungsprotokoll nennt exakten Befehl, Arbeitsverzeichnis, Datenpfad, Versionsstand, Umgebung ohne Secrets, Start-/Endzeit, Exitcode und Ergebnis. Dependencies nicht unbegründet aktualisieren; ein reproduzierbarer vorhandener Build ist zuerst aussagekräftiger. Fehlende Tools oder Abhängigkeiten getrennt von Produktfehlern berichten.

Prüfwellen: zuerst vorhandene gefahrlose Suites in isolierten Kopien; dann die unabhängigen M-/E-Grenzfälle; dann gezielte Robustheits-/Concurrencytests; schließlich optional autorisierte Live-Kalibrierung. Reale LLM-Regressionen sind getrennt, kostenpflichtig und nicht automatisch durch „alle Tests“ autorisiert. Ein übersprungener Live-Test bleibt sichtbar.

Gezielte neue Reviewtests sind zulässig, wenn sie eine konkrete Fehlerhypothese, mathematische Invariante oder externe Contractlücke prüfen. Keine massenhaften Tests, die nur aktuelle Implementierung reproduzieren. Diagnosecode gehört in den Review-Arbeitsbereich, nicht unbemerkt in das Produkt.

24. Priorisierung: Auswirkung und Sicherheit getrennt

Priorität	Maßstab	Beispiele, deren tatsächliche Wirkung nachzuweisen ist
P0	Unmittelbar schwerer Daten-/Sicherheits-/Bewertungsfehler; sofortiger Handlungsbedarf	Falsche positive Freigabe trotz nachgewiesenem Grenzbruch; irreversible Datenzerstörung; belastbar offengelegte produktive Secrets
P1	Wesentlicher fachlicher oder betrieblicher Fehler in realistischem Ablauf	Falscher DD-/Rendite-Nenner, stille Historienverkürzung, unzuverlässiges Resume, häufig scheiternde Kataloge
P2	Relevanter begrenzter Fehler oder belegtes Zuverlässigkeits-/Performanceproblem	Irreführende Freshness, fehlender Fehlerkontext, teure Wiederholungsverarbeitung, uneinheitlicher Export
P3	Wartbarkeit, Präzision, Dokumentation oder geringere Bedienprobleme	Veraltete Schwellenbeschreibung, unnötige Kopplung, inkonsistente Labels ohne aktuelle Bewertungswirkung

Priorität ist unabhängig vom Evidenzstatus. Status pro Punkt: **bestätigter Fehler**, **bestätigte Dokumentationsabweichung**, **begründetes Risiko**, **offene Hypothese**, **Verbesserung ohne Fehlernachweis**, **historisch behoben** oder **widerlegt**. Ergänze Sicherheit hoch/mittel/niedrig mit Grund. Ein sehr gefährlicher denkbarer Fall ohne Reproduktion bleibt ein Risiko/Hypothese, kein bestätigter P0-Produktfehler.

Bewerte betroffene Projekte/Quellen/Signale, Eintrittsbedingungen, Datenfenster, Auswahl-/Ampelwirkung, Recovery und Beobachtbarkeit. Ein minimales Gegenbeispiel beweist einen Fehler unter diesen

Bedingungen; es beweist nicht dessen Verbreitung im gesamten Bestand. Quantifiziere Verbreitung erst nach Bestandprüfung mit benanntem Nenner.

25. Verbindliches Befundformat

Jeder Hauptbefund bekommt eine stabile ID wie SDR-001. Verwende dieses Format:

1. **Titel, Priorität, Status, Sicherheit.** Ein konkreter Satz beschreibt das tatsächliche Problem.
2. **Betroffene Projekte/Abläufe.** Producer, Consumer, Sichtbarkeit und Grenzen.
3. **Sollverhalten und Grundlage.** Aktuelle Nutzerregel, fachliche Definition, Contract oder belegte technische Anforderung.
4. **Istverhalten und Belege.** Datei/Funktion/Zeile, Logereignis, Snapshot/Hash, Repro-/Testreferenz.
5. **Reproduktion.** Exakte Eingabe, Vorbereitung, Befehl/Interaktion, Erwartung und beobachtetes Resultat. Keine echten Secrets.
6. **Ursache.** Erster ursächlicher Fehler, Folgefehler und alternative Erklärung.
7. **Auswirkung.** Zahlen-/Ampel-/Daten-/Betriebsfolge, erreichbarer Nutzerpfad und nachgewiesener Umfang.
8. **Stärkste Gegenposition.** Warum es eventuell korrekt/gewollt sein könnte und wie die Evidenz diese Position bestätigt oder widerlegt.
9. **Maßnahme.** Minimaler Fixvorschlag, robuste längerfristige Alternative, Tradeoffs und betroffene Verträge. Kein produktiver Patch in diesem Auftrag.
10. **Abnahme.** Konkrete Regressionstests, erwartete Sollwerte und notwendige Rescans/Migration/Reportinvalidierung.
11. **Aufwand und Abhängigkeiten.** Klein/mittel/groß mit Annahmen, Reihenfolge, Reviewbedarf und Risiken; keine Scheingenauigkeit in Stunden.
12. **Offene Grenzen.** Nicht ausgeführte Liveprüfung, fehlende Daten/Instrumente, Zeitraum oder externe Konsumenten.

Logs und Codezitate kurz und zielgerichtet halten; vertrauliche Werte redigieren. Bei einem ausschließlich statisch bewiesenen Fehler begründen, warum der konkrete erreichbare Pfad ausreicht. Bei Nicht-Reproduktion den Status herabstufen und nicht durch plausiblen Text ersetzen.

Die maschinenlesbare `findings.json` muss mindestens ID, Titel, Priorität, Status, Sicherheit, Projekte, Belege, Reproduktion, Auswirkung, Maßnahme, Abnahme und Abhängigkeiten enthalten. Kein Pflichtfeld mit erfundenem Inhalt füllen; fehlende Evidenz explizit `null`/offen nennen.

26. Verbesserungsplan mit begründeter Reihenfolge

Leite Maßnahmen aus bestätigten Befunden und quantifizierten Risiken ab. Gruppiere sie in:

- **Sofort:** konkrete Schutz-/Korrekturmaßnahmen für belegte schwere Fälle; erforderliche Rescans und Kennzeichnung bereits betroffener Berichte.
- **Nächste Iteration:** Daten-/Semantikvertrag, vollständige Fehlerzustände, gezielte Regressionen, konsistente Invalidation und Recovery.
- **Danach:** Performance, gemeinsame Contract-Fixtures, modulare Entkopplung, Build-/Betriebsreproduzierbarkeit und dokumentierte Architektur.

Für jede Maßnahme nenne Nutzen, Belegbezug, Voraussetzungen, Aufwandklasse, mögliche Nebenwirkungen, Abnahmekriterium und Reihenfolge. Bewertungsregeln nicht „verbessern“, indem harte fachliche Anforderungen abgeschwächt werden. Zwischen Fehlerbehebung, Produktentscheidung und Architekturwunsch unterscheiden.

Beispielhafte Entscheidungsfragen: explizite DD-Methode im REST-Vertrag versus kurzfristige Feldumbenennung; Composite-Identität versus sichere sichtbare Kollisionabwehr; einheitlicher Daten-Snapshot versus nur zusätzliche SHA-Felder; gemeinsame Storebibliothek versus gemeinsame Tests; echte historische Equity-Daten versus ehrliche Unbekanntkennzeichnung. Vergleiche jeweils stärkste Alternative und Migrationskosten.

Eine Maßnahmenliste ohne verknüpfte Befunde ist keine belastbare Roadmap. Vorschläge dürfen nicht sämtliche Quellen zu einem Framework zwingen, wenn Swing, JavaFX, Python oder Plattformverträge verschiedene Anforderungen haben.

27. Geforderte Review-Artefakte und Berichtsgliederung

Erzeuge unter `SignalKiScanner/doc/reviews/gesamtworkspace_<tatsächliches_reviewdatum>/` einen neuen Reviewbereich; überschreibe dieses Konzept nicht. Neue Reviewprotokolle bleiben dort. Archivierung nach `waste/` erst nach tatsächlich vollständiger Umsetzung, nicht nach Fertigstellung des Analyseberichts.

Verpflichtende Ergebnisse:

- `REVIEW_BERICHT.md`: vollständiger, KI-lesbarer Hauptbericht mit Zusammenfassung, sieben Projektkapiteln, Querverbindungen, Befunden, Maßnahmen und Grenzen.
- `output/pdf/REVIEW_BERICHT.pdf`: inhaltsgleiche druck-/lesefähige Fassung mit Inhaltsverzeichnis, Seitenzahlen, lesbaren Tabellen und Quellenindex.
- `ABDECKUNG.md`: Projekte, Module, Abläufe, Logs, Daten/Tests; Prüfstatus und konkrete Lücken.
- `EVIDENZINDEX.md`: Manifest, Code-/Logbelege, Repros, Tests, Hashes und redigierte Ergebnisverweise.
- `findings.json`: strukturierte Maßnahmen-/Befundliste für Folgebearbeitung.

Der Hauptbericht muss diese Inhalte enthalten:

1. Entscheidungsübersicht: bestätigte schwerste Fehler, dringendste Maßnahmen, belegte Grenzen.
2. Scope/Arbeitsstand: sieben Projekte, neue Module, uncommittete Änderungen und externe Anschlussprüfungen.
3. Tatsächliche Architektur mit Datenfluss-, Thread-/Job- und Contractdarstellung.
4. Aktuelle Regeln und Kennzahl-/Basis-/Zeitraummatrix.
5. Je Projekt: Rolle, Code-/Ablaufprüfung, Logmuster, Teststand, Befunde, Verbesserungen und offene Fragen.
6. Quellen-zu-Hub-/Hub-zu-Konsumenten-Befunde und identitäts-/semantikübergreifende Fehlerketten.
7. Mathematische Nachrechnungen und E2E-Gegenbeispiele mit Soll/Ist.
8. Betriebs-, Sicherheits-, Performance- und Wartbarkeitsbewertung mit konkreten Belegen.
9. Priorisierte Maßnahmen mit Abnahme, Migration/Rescan und Gegenposition.
10. Vollständiger Befundkatalog sowie Evidenz-, Test- und Abdeckungsanhang.

Die Zusammenfassung darf nur Aussagen enthalten, die im Befundteil belegt sind. Projekte ohne bestätigte Fehler erhalten trotzdem ihre geprüften Bereiche, Grenzen und Verbesserungsvorschläge; keine erfundenen

Fehler, um alle Kapitel zu füllen. „Keine Fehler gefunden“ heißt nur innerhalb des ausdrücklich geprüften Umfangs.

PDF und Markdown müssen dieselben Befund-IDs, Prioritäten, Werte, Projektkapitel und Einschränkungen enthalten. PDF-Layout prüfen: keine abgeschnittenen Tabellen, winzigen Schriften, fehlenden Umlaute oder schwarzen Ersatzglyphen. Wichtige Diagramme in beiden Formaten verständlich darstellen. Die PDF soll durchsuchbar sein, keine reine Bildsammlung. Quellen und Reproartefakte müssen vom Hauptbericht auffindbar sein.

28. Abschlusskriterien und Startanweisung

Der Review ist erst abgeschlossen, wenn alle sieben Projekte und neu gefundenen produktrelevanten Module in der Abdeckungsmatrix stehen; zentrale Berechnungen unabhängig nachgerechnet sind; kritische Verträge über Producer/Consumer geprüft sind; alle auffindbaren relevanten Logs inventarisiert und die tatsächliche Analysetiefe ausgewiesen ist; wichtige Befunde eine Gegenprüfung haben; ausgeführte und nicht ausgeführte Tests getrennt sind; Maßnahmen konkrete Abnahmen besitzen; MD/PDF vollständig und geprüft vorliegen.

Offene Liveprüfungen und externe Konsumenten verhindern keinen Bericht, solange sie deutlich benannt sind. Sie verhindern aber die Behauptung vollständiger Live- oder Consumer-Verifikation. Wenn Zeit-/Kontextgrenzen auftreten, führe den Review mit persistentem Manifest, Prüfstatus und Evidenzindex fort; ersetze ihn nicht durch eine kurze allgemeine Checkliste.

Kurzer Startprompt zur Verwendung zusammen mit dieser Datei:

Lies REVIEW_KONZEPT_UND_MASTERPROMPT.md vollständig und führe das darin spezifizierte Codereview aller sieben Projekte im Workspace D:/AntiGravitySoftware/GitWorkspace/SIGNALDOWNLOADER aus. Prüfe den aktuellen Arbeitsbaum einschließlich uncommitteter Änderungen. Plane zuerst, arbeite parallel mit Subagents, verifiziere mathematische Sollwerte unabhängig und untersuche Code, komplette relevante Abläufe sowie alle auffindbaren relevanten Logs. Erzeuge den vollständigen Fehler-/Verbesserungsbericht als Markdown und inhaltsgleiches PDF samt Abdeckungsmatrix, Evidenzindex und findings.json. Keine Produktivfixes oder Liveaktionen; isolierte Repros und Tests sind Teil des Auftrags. Trenne belegte Fehler, Risiken, historische Fälle und offene Fragen. Beginne jetzt mit dem Manifest und führe alle unabhängig ausführbaren Prüfungen bis zu den geprüften Berichtsartefakten durch.

29. Quellenindex der Konzepterstellung

Alle Pfade beziehen sich auf den angegebenen Workspace, soweit kein anderer Pfad genannt ist. Die Anker wurden bei der lesenden Bestandsaufnahme geprüft; beim späteren Review müssen Zeilen und Zustand neu validiert werden.

Kontext / Prüfschwerpunkt	Konkrete Ausgangsquelle
Sieben Projekte, Rollen, Ports	Workspace README.md, SignalKiScanner/doc/21_megaprojekt-architektur.md
Gemeinsame Monitorbedienung	Workspace UI_KONVENTIONEN.md
Service-/Prozess-/Readinessregeln	Workspace startall.bat:15, :26, :50; Scanner start.bat:25, :28, :33
Aktuelle datierte Fachregeln	Scanner AGENTS.md, vollständig; aktuelle Ergänzungen bis 04.10.2026
DD-Produzenten und Empfänger	Scanner src/mqlkiscanner/ingest.py:291, :320; pipeline.py:276, :286, :573
RetDD und Kapitalbasis	Scanner portfolio_statistik.py:94 unter src/mqlkiscanner/; pipeline.py:294; ampel_matrix.py:356
Persistenz und ID-Kollisionen	Scanner src/mqlkiscanner/db.py:201, :314, :716
Cache/Pagination/Atomic write	Scanner src/mqlkiscanner/ingest.py:52, :144, :197, :251
Ergebnisweitergabe	Scanner src/mqlkiscanner/rest_api.py:136; tradeserver_sync.py:60 im selben Paket
Testauswahl und Runtime	Scanner pytest.ini, requirements.txt; sechs Java-Projekte jeweils pom.xml
Historische Regelkorrekturen	waste/retddequity_2026-10-03/review.md, waste/drawdown_3signale_2026-10-03/, waste/laufreview_2026-10-02/
Scanner-Laufzeitfehler/-grenzen	Scanner data/scan_workflow.log:4597, :4660, :4679
Pelican DD-/FX-Vertrag	PelicanTrading src/main/java/de/pelicanmonitor/rest/RestApiServer.java:377, :544; zugehörige report/ReportService.java und EquityKurve.java
WinnerLooser API-/Fortsetzungsstand	PelicanWinnerLooser README.md, doc/API_VERIFIKATION.md, Auth-/Crawl-/Analytics-Code, data/pelican-winner-looser.log
Robo Aggregation / offene Reste	roboforex src/main/java/de/robomonitor/deals/TradeAggregator.java:63, :126
Robo DD und Nettoexport	roboforex src/main/java/de/robomonitor/model/EquityKurve.java:20; report/SignalPayloadBuilder.java:44 und rest/RestApiServer.java:320, :380 im Package de/robomonitor
Vantage DD / API-Normalisierung	vantage src/main/java/de/vantagemonitor/report/SignalPayloadBuilder.java:43; api/VantageClient.java:357, :404 im Package de/vantagemonitor; download_trades.py

Kontext / Prüfschwerpunkt	Konkrete Ausgangsquelle
Zulu DD / LLM	zulumonitor src/main/java/de/zulumonitor/report/TradePayloadBuilder.java:39; ReportService.java:77 im selben Reportpackage
WinnerLooser Arbeitsplan / FX	PelicanWinnerLooser src/main/java/de/pelicanwinnerlooser/crawl/CrawlService.java:117, :337; analytics/WaehrungsUmrechnung.java:125 im Package de/pelicanwinnerlooser
MQL Originaldownload / MPDD	MqlDownloader src/downloader/SignalDownloader.java:1113, :1303; src/calculators/MPDDCalculator.java:42; src/converter/HtmlConverter.java:167
Vantage Katalogfehler	vantage data/vantagemonitor.log:483, Katalogclient und RestApiServer.java unter src/main/java/
Zulu Host/Marker/Reports	zulumonitor data/zulumonitor.log:1087, TradeListenStore/REST/ReportService unter src/main/java/
Externer MQL-Laufzeitort	C:/Forex/MqlAnalyzer/logs/application.log, zugehörige MqlDownloader-Pfad-/Loggerkonfiguration

Grenze des Quellenindex: Er ist eine Einstiegshilfe für das Konzept, kein Nachweis der Vollständigkeit des späteren Reviews. Aktuelle Gesamt-Testpassraten, Bestandsauswirkungen der Prüfhypothesen und vollständige Liveabläufe wurden hier nicht festgestellt.